
Signal Processing

Übersicht

Aufgabe	2
Infrared-Controlled Timer	4
Bewertungsinformationen	5
Kompilierung	5
Code for Humans (Bonuspunkte für die jeweilige Aufgabe)	5
Magic Numbers	5
Single Responsibility (SRP)	5
Aussagekräftige Benennung	5
Formatierung und Einrückung	6
Literaturverzeichnis	7

Aufgabe

In dieser Übungseinheit sollen Sie mithilfe einer Infrarotfernbedienung Ihrem Controller Signale übertragen. Diese sollen von diesem mittels eines Infrarotempfängers empfangen werden. Der Empfänger wird wie folgt angeschlossen:

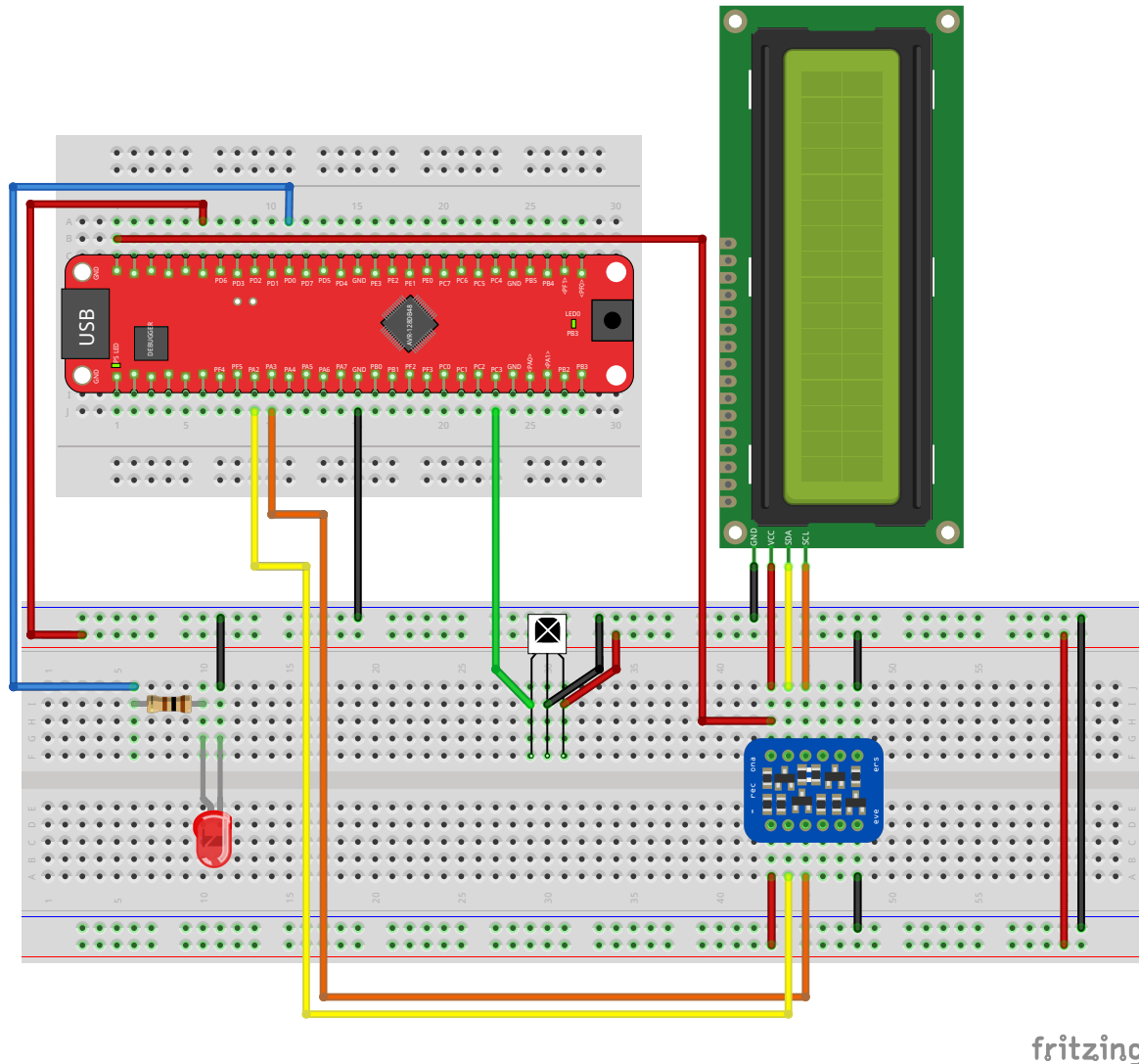


Abbildung 1: Das TXB0104-Bauteil stammt aus der Adafruit Fritzing Library [1]

Mit der Infrarotfernbedienung können Signale moduliert in 38kHz gesendet werden, auf welche der Empfänger reagiert. Wird Infrarotstrahlung mit dieser bestimmtem Frequenz empfangen, wird die Signalleitung des Empfängers auf LOW gezogen. Ist dies nicht der Fall, wird HIGH ausgegeben. Die Codierung des Signals entspricht dem [NEC Protocol](#).

Das der Fernbedienung generierte Signal kann folgendermaßen aussehen:

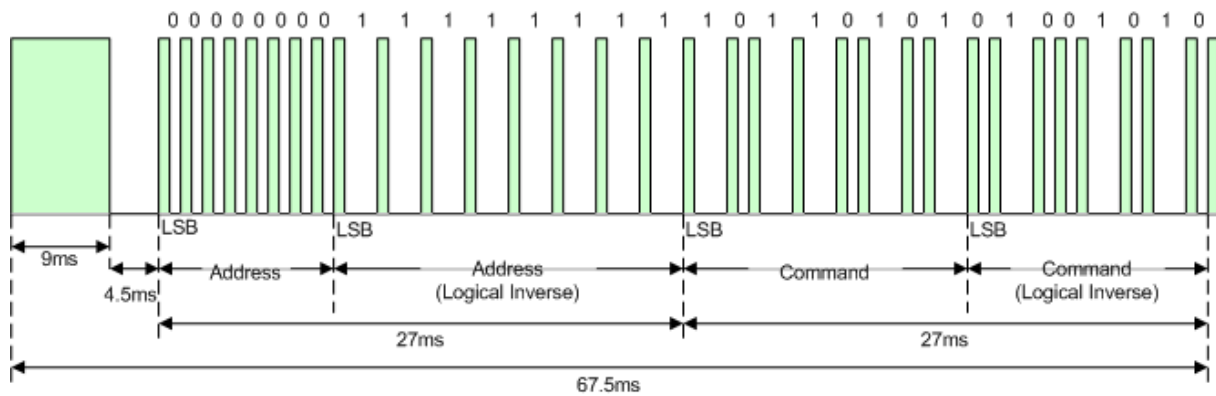


Abbildung 2: https://valhalla.altium.com/Learning-Guides/Legacy/CR0183%20WB_IRCODER%20Infrared%20Encoder.PDF, NEC Frame

Dabei stehen die grünen Impulse für ein Senden des Infrarotlichts.

Ein Signalverlauf der Signalleitung des Empfängers kann wie folgt aussehen

nicht das Command-Gegenstück zum abgebildeten NEC-Frame!



Abbildung 3: IR - Code 0xE2 / Taste B

Um aus diesem Signal Informationen zu extrahieren, müssen Sie auf die Start-Kondition (9ms LOW, 4.5ms HIGH) achten und den **Command**-Bereich des Signals abtasten.

Die Fernbedienung kann folgende Commands senden:

Binary	Hexcode	Decimal	Button
0b0100 0000	0x40	64	X
0b0100 0110	0x46	70	UP
0b0001 0101	0x15	21	DOWN
0b0100 0100	0x44	68	LEFT
0b0100 0011	0x43	67	RIGHT
0b0101 1010	0x45	69	A
0b0100 0111	0x47	71	B
0b0000 1001	0x09	9	C
0b0000 0111	0x07	7	0
0b0001 0110	0x16	22	1
0b0001 1001	0x19	25	2
0b0000 1101	0x0D	13	3
0b0000 1100	0x0C	12	4
0b0001 1000	0x18	24	5
0b0101 1110	0x5E	94	6
0b0000 1000	0x08	8	7
0b0001 1100	0x1C	28	8
0b0101 1010	0x5A	90	9

Infrared-Controlled Timer

Nutzen Sie die Flankenerkennung mittels Interrupts in Kombination mit einem Timer als Zeitmesser zwischen den Flanken, um aus einem gesendeten Signal den COMMAND-Code zu extrahieren. Nachdem Sie diese Codes empfangen und interpretieren können, soll damit der Timer mit LCD-Anzeige gesteuert werden.

Mit den Commands 0 - 9 oder UP bzw. DOWN soll der Timerwert eingegeben bzw. um 1s erhöht/verringert werden.

Mit dem Betätigen der X-Taste soll der Timer gestartet/gestoppt werden können.

Nach Ablauf des Timers soll die LED an PD0 leuchten.

Bewertungsinformationen

Kompilierung

Die Kompilierung der Aufgabe erfolgt mit dem Befehl

```
{Microchip-Toolchain Installationsverzeichnis}/xc8/v3.10/bin/xc8-cc -Wall \  
-Wextra -O1 -mcpu=AVR128DB48 \  
-mdfp={Ort auf Ihrem PC}/.mchp_packs/Microchip/AVR-Dx_DFP/2.7.321/xc8 \  
-I./Includes/AVR128DB48_I2C/ \  
-I./Includes/I2C_LCD/ -o main1.elf ./Includes/I2C_LCD/I2C_LCD.c \  
./Includes/AVR128DB48_I2C/AVR128DB48_I2C.c main1.c \  
-Wl,-Map=program1.map \  
-Wl,--gc-sections
```

Sollte Ihr Code nicht kompilieren wird die jeweilige Teilaufgabe mit **0 Punkten bewertet**. Für jede Warning werden **0.5 Punkte abgezogen**.

Stellen Sie also sicher, dass der Code zum einen **kompiliert** und alle Warnings entfernt werden.

Code for Humans (Bonuspunkte für die jeweilige Aufgabe)

Guter Code funktioniert nicht nur, er ist auch lesbar. Für eine besonders saubere Implementierung (Stichwort: Clean Code) vergeben wir zusätzliche Bonuspunkte. Das ist Ihre Chance, kleine Fehler in der Logik durch gute Struktur und Wartbarkeit auszugleichen.

- **Regelung:** Die Bonuspunkte werden zum Gesamtergebnis addiert.
- **Limit:** Die Endwertung der Aufgabe kann 100 % nicht überschreiten.

Kriterium	Punkte	Erfüllt(x1)	Teilweise erfüllt(x0,5)
Es befinden sich keine <i>Magic Numbers</i> im Code	2		
Funktionen erfüllen (so gut es geht) nur eine Aufgabe	1		
Namen von Funktionen und Variablen sind sinnvoll gewählt	1		
Konsistente Formatierung und Einrückung	1		

Magic Numbers

Zahlenwerte sollten nicht ohne Kontext im Code stehen (z. B. `if (status == 4)`). Solche „magischen“ Zahlen machen den Code schwer wartbar. Stattdessen sollte Konstanten oder Enums mit sprechenden Namen (z. B. `if (status == STATUS_READY)`) genutzt werden. Mehr dazu kann [hier](#) nachgelesen werden.

Single Responsibility (SRP)

Eine Funktion sollte genau eine logische Aufgabe erfüllen. Wenn eine Funktion beispielsweise gleichzeitig Sensordaten liest, eine komplexe Berechnung durchführt **und** ein Display aktualisiert, sollte sie aufgeteilt werden. Mehr dazu kann [hier](#) nachgelesen werden.

Aussagekräftige Benennung

Verwenden Sie keine Namen wie `a`, `temp` oder `func1()`. Namen sollten beschreiben, was die Variable enthält oder was die Funktion tut (z. B. `current_temperature_celsius` statt `val`). Mehr dazu kann [hier](#) nachgelesen werden.

Formatierung und Einrückung

Für konsistente Einrückung und Formatierung empfiehlt es sich einen *Formatter* zu verwenden (Bspw. [clang-format](#))

Literaturverzeichnis

- [1] Adafruit Industries, „Adafruit Fritzing Library“. [Online]. Verfügbar unter: <https://github.com/adafruit/Fritzing-Library>